

CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

PancakeV3Factory

0x0BFbCF9fa4f9C56B0F40a671Ad40E0805A091865

BNB Smart Chain

Compiler: v0.7.6+commit.7338295f

Date: October 7, 2026

OVERALL RISK RATING AI-assisted audit. Not a substitute for manual review.

Medium

Executive Summary

Independent security review of publicly verified source, not commissioned by the project. PancakeV3Factory is a largely standard fork of the Uniswap V3 Factory with an added whitelist/tick-spacing-extra-info system and LM pool deployer role. No critical fund-draining bugs were found in the reviewed logic, but the contract carries significant centralization risk (owner can arbitrarily collect protocol fees and gate pool creation), lacks zero-address protection on privileged setters, and its constructor-based initialization is inconsistent with the 'upgradeable' deployment pattern reported for this address. These issues should be reviewed and hardened, particularly around owner key custody (multisig/timelock) and initialization safety if the contract is indeed deployed behind a proxy.

Findings (6)

F-001

Medium

Constructor-based initialization is incompatible with reported upgradeable deployment

contracts/PancakeV3Factory.sol:43–57

DESCRIPTION

The contract metadata indicates this instance is deployed as an upgradeable (proxy-backed) contract, yet PancakeV3Factory.sol uses a plain Solidity constructor to set `owner`, `poolDeployer` (immutable), and default fee tiers. Constructors only execute in the context of the implementation contract's own deployment transaction, they do NOT run in the proxy's storage context when the implementation is delegatecalled through a proxy. There is no `initialize()` function, no `Initializable` guard, and no `\_disableInitializers()` call in the implementation constructor.

IMPACT

If this implementation is ever wired behind a proxy (as the 'Upgradeable: yes' flag suggests), proxy storage for `owner` would default to `address(0)` and none of the default fee tiers would be set, effectively bricking the factory or leaving it in an uninitialized state that anyone could attempt to claim/initialize if an initializer were later added without protection. It also exposes the implementation contract itself to being initialized/front-run directly since there is no `\_disableInitializers()` lock.

RECOMMENDATION

If the contract is genuinely intended to be non-upgradeable (standard constructor deployment, which matches the on-chain PancakeSwap V3 factory design), correct the 'Upgradeable' classification in deployment metadata/documentation. If a proxy is actually used, refactor to the standard upgradeable pattern: inherit `Initializable`, replace the constructor body with an `initialize()` function protected by `initializer`, and add a constructor that calls `\_disableInitializers()` to lock the logic contract.

```
constructor(address _poolDeployer) {
    poolDeployer = _poolDeployer;
    owner = msg.sender;
    ...
}
```

F-002

Medium

Owner has unilateral, unrestricted ability to withdraw accumulated protocol fees from any pool

contracts/PancakeV3Factory.sol:134–141

DESCRIPTION

`collectProtocol` and `setFeeProtocol` allow the single `owner` address to redirect every pool's accumulated protocol fees to an arbitrary `recipient` and to change the protocol fee cut at will, with no timelock, multisig enforcement, or on-chain governance delay visible in this contract.

IMPACT

A compromised or malicious owner key can immediately redirect all accumulated protocol fees across every PancakeSwap V3 pool to an attacker-controlled address, and can raise the protocol fee percentage to its maximum to siphon future swap fees, all without any user-facing delay or veto mechanism.

RECOMMENDATION

Route `owner` through a timelock controller and/or multisig (e.g., Gnosis Safe + TimelockController) so privileged fee-collection and fee-protocol changes are subject to a public delay, and emit sufficiently detailed events (already partially done) to allow off-chain monitoring/alerting.

```
function collectProtocol(
    address pool,
    address recipient,
    uint128 amount0Requested,
    uint128 amount1Requested
) external override onlyOwner returns (uint128 amount0, uint128 amount1) {
    return IPancakeV3Pool(pool).collectProtocol(recipient, amount0Requested, amount1Requested);
}
```

F-003

Low

Missing zero-address validation in setOwner

contracts/PancakeV3Factory.sol:86–89

DESCRIPTION

`setOwner` allows the current owner to set `\_owner` to `address(0)` with no validation.

IMPACT

An accidental or malicious call with `\_owner == address(0)` permanently disables every `onlyOwner` function in the factory (fee tier management, whitelist management, protocol fee collection, LM pool deployer assignment) with no recovery path, since there is no two-step ownership transfer or recovery mechanism.

RECOMMENDATION

Add a zero-address check, and consider implementing a two-step ownership transfer pattern (propose/accept) similar to OpenZeppelin's `Ownable2Step` to avoid irrecoverable mistakes.

```
function setOwner(address _owner) external override onlyOwner {
    require(_owner != address(0), "zero address");
    emit OwnerChanged(owner, _owner);
    owner = _owner;
}
```

F-004

Low

Missing zero-address validation in setLmPoolDeployer

contracts/PancakeV3Factory.sol:131–133

DESCRIPTION

``setLmPoolDeployer`` sets the ``lmPoolDeployer`` address without validating it is non-zero.

IMPACT

Setting ``lmPoolDeployer`` to ``address(0)`` is low-impact on its own (it simply disables the ``onlyOwnerOrLmPoolDeployer`` alternate path, falling back to owner-only), but it is inconsistent with defensive coding practice and could mask configuration errors.

RECOMMENDATION

Add ``require(_lmPoolDeployer != address(0), "zero address");`` before updating state.

```
function setLmPoolDeployer(address _lmPoolDeployer) external override onlyOwner {
    lmPoolDeployer = _lmPoolDeployer;
    emit SetLmPoolDeployer(_lmPoolDeployer);
}
```

F-005 · SWC-103

Low

Floating pragma in interface files

contracts/interfaces/IPancakeV3Factory.sol:2–2

DESCRIPTION

All reviewed interface files declare ``pragma solidity >=0.5.0;``, which permits compilation with any compiler version 0.5.0 and above, including future major versions with breaking semantic changes.

IMPACT

Floating pragmas can cause interfaces to be compiled with a compiler version different from the one used for the implementation contract (``=0.7.6``), risking ABI-selector mismatches or unexpected compiler behavior changes in downstream integrations that import these interfaces directly.

RECOMMENDATION

Pin interface pragmas to the same version used by the implementation, e.g. ``pragma solidity =0.7.6;``, or constrain the range explicitly (e.g. ``>=0.5.0 <0.8.0``).

```
pragma solidity >=0.5.0;
```

F-006 · SWC-107

Informational

External call to pool deployer occurs before state update in createPool

contracts/PancakeV3Factory.sol:69–75

DESCRIPTION

``createPool`` calls ``IPancakeV3PoolDeployer(poolDeployer).deploy(...)`` (an external call to an immutable, trusted contract) before writing the resulting address into the ``getPool`` mapping, deviating from strict checks-effects-interactions ordering.

IMPACT

Because ``poolDeployer`` is immutable and set at construction by the factory owner (trusted), and `tokenA/tokenB` are never called directly in this function, there is no practical reentrancy vector today. This is purely a defense-in-depth / code-pattern observation.

RECOMMENDATION

No functional change required given the trusted, immutable deployer; optionally add a comment documenting the trust assumption for future auditors/maintainers.

```
pool = IPancakeV3PoolDeployer(poolDeployer).deploy(address(this), token0, token1, fee,
tickSpacing);
getPool[token0][token1][fee] = pool;
getPool[token1][token0][fee] = pool;
```

Gas Optimizations

Use external instead of public for owner-only setter functions

contracts/PancakeV3Factory.sol:92–120

``enableFeeAmount``, ``setWhiteListAddress``, and ``setFeeAmountExtraInfo`` are declared ``public`` but never called internally; mark them ``external`` to save a small amount of gas on calldata handling.

Cache feeAmountTickSpacingExtraInfo struct reads

contracts/PancakeV3Factory.sol:63–67

``info`` is already cached via memory copy in `createPool`, which is good; ensure similar caching pattern is applied consistently if additional reads of the same mapping slot are added in future changes.

Duplicate reverse-pool mapping write could be combined with a helper

contracts/PancakeV3Factory.sol:73–75

Consider using a single internal ``_setPool`` helper for the two symmetric ``getPool`` writes to reduce code duplication (marginal gas/readability improvement, no functional change).

Appendix - Metadata

Contract	PancakeV3Factory
Address	0x0BFbCF9fa4f9C56B0F40a671Ad40E0805A091865
Chain	BNB Smart Chain
Compiler	v0.7.6+commit.7338295f
Audit ID	portfolio-pancakeswap-v3-factory
Date	10/7/2026

Files analyzed

```
[user] contracts/interfaces/IPancakeV3Factory.sol
[user] contracts/interfaces/IPancakeV3Pool.sol
[user] contracts/interfaces/IPancakeV3PoolDeployer.sol
[user] contracts/interfaces/pool/IPancakeV3PoolActions.sol
[user] contracts/interfaces/pool/IPancakeV3PoolDerivedState.sol
[user] contracts/interfaces/pool/IPancakeV3PoolEvents.sol
[user] contracts/interfaces/pool/IPancakeV3PoolImmutables.sol
[user] contracts/interfaces/pool/IPancakeV3PoolOwnerActions.sol
[user] contracts/interfaces/pool/IPancakeV3PoolState.sol
[user] contracts/PancakeV3Factory.sol
```

MANUAL AUDIT UPGRADE

Need a deeper audit?

\$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

[Request Manual Audit '](https://cryptohawking.com/audit/manual?ref=portfolio-pancakeswap-v3-factory)

<https://cryptohawking.com/audit/manual?ref=portfolio-pancakeswap-v3-factory>

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

<https://cryptohawking.com/services>

WhatsApp (preferred): +92 322 4274236, <https://wa.me/923224274236>